

UTXO vs Account Model

23.08.2023

Overview

Blockchain have giant common shared ledgers .The format in which you store and maintain the ledger to deal with the transactions gives different properties to blockchain.

The two most common ways or the most popular ways to format the Ledger are

- UTXO Model
- Account Model

Utxo is used by bitcoin and accounts used by ethereum.

UTXO MODEL

The word UTXO stands for Unspent Transaction Output Model.It represents the amount of digital currency remaining after a transaction has been executed.

The movement of assets is recorded as a directed acyclic graph (DAG) between addresses.Directed acyclic graphs don't allow circular relationships between nodes.

The graphic above shows a directed acyclic graph of the UTXO model on the left.In the UTXO model, the entire graph of transaction outputs, spent and unspent, represents the global state

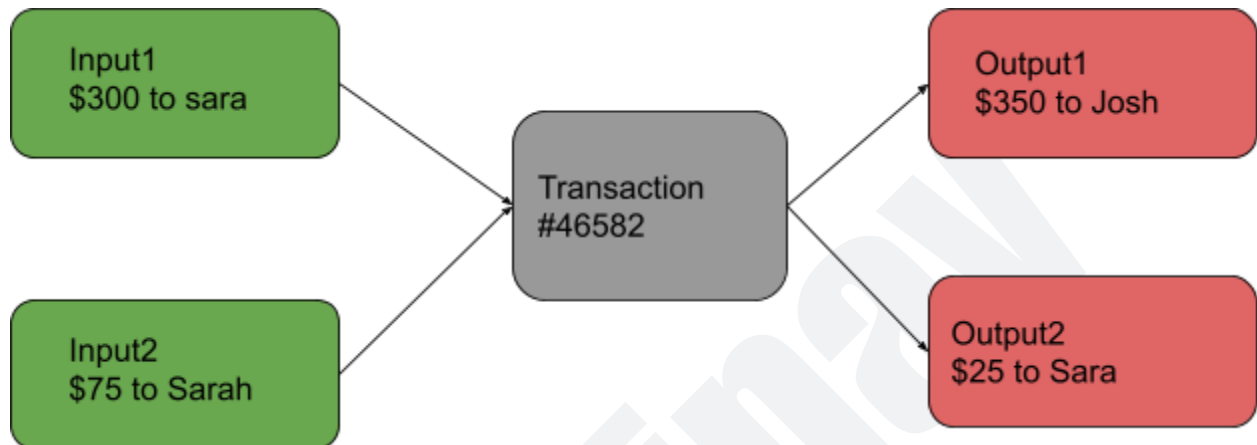
Simplified Explanation

In the UTXO MODEL for every unspent transaction we need inputs Which is where your reference unspent transaction outputs that you have access to and outputs which is where you show who you are sending the money to .

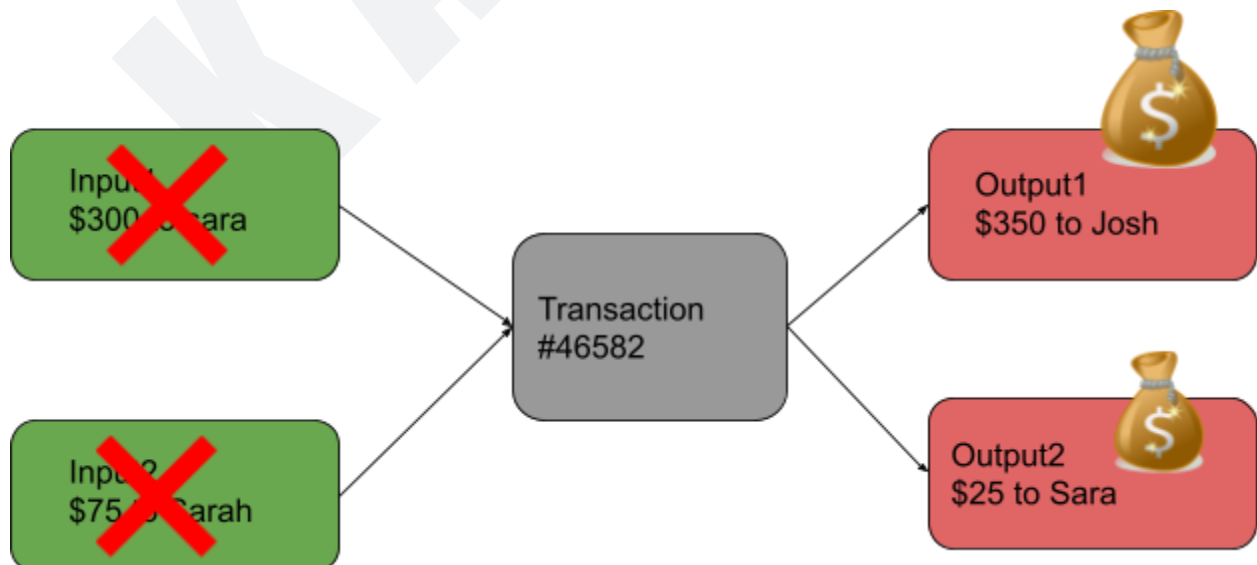
For example, consider Jane,Daniel,Josh and Sara with ₹500,150,10,0

So Sara Susara received ₹300 from Jane and ₹75 from Daniel so with this money Sara pays Josh 350\$ and keeps the remaining 25\$ for herself.

In this example we can see that the inputs are the 300 \$ and 75 \$ that Sara received Previously and the outputs are the 350\$ sent to Josh and 25 \$ that Sara gets back.



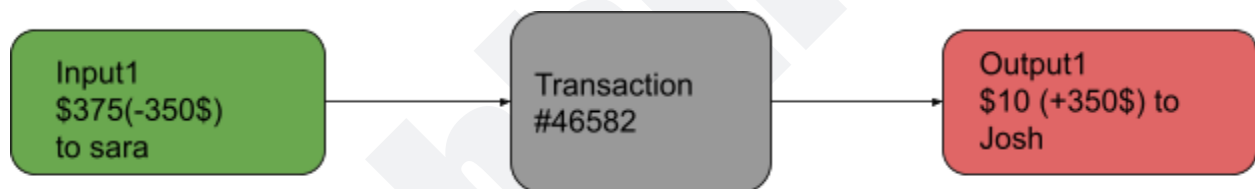
UTXO Model is similar to using Cash Where each unspent transaction output Can only be spent all at once. You can't spend half or a quarter of an UTXO at a time. In this example sarah puts in \$375 even though she only needed to send Josh \$350. This is like if she had a \$300 bill and \$75 bill. During the transaction there are \$300 and \$75 bills get destroyed and a new \$350 bill is created and sent to Josh as well as \$25 bill that gets sent back to Sarah.



Here lies the security aspect, The \$25 that gets sent back to Saha she doesn't actually have to use the Same address that she sent to the transaction from. She can easily create a new address In her wallet and send it there instead. This can help a bit with privacy As you noticed that we won't have a single address associated with you where all your funds are stored.

Account Model

The account based model is much simpler: every address has a balance and when you make a transaction you simply say you want to send a certain amount from one address to another this leads to a smaller transaction size which is more efficient.



For example consider the previous situation where Sara has \$375 in her account she pays josh \$350 she now has \$25 in her account and josh who started with \$10 will now have \$360 in his account. This is like a normal bank account where everyone has their own account And you can send any amount to each other at a time this account based model is far simpler On the surface but both models have their advantages and disadvantages which we are going to go through.

Global State

An important deduction is to be made that how utxo handles the global state compared to the account model by state means the way the up to date list of balance on the ledger is maintained .

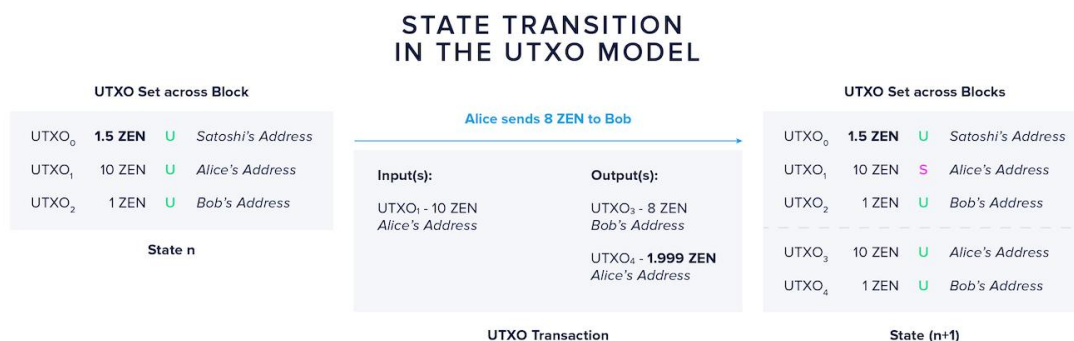
As the state is something shared among thousands of people. Precautions are to be made so that when changing it to make sure that we aren't changing something someone else has already changed. There are three main functions strange state removing a row, updating a row and adding a row when removing a row you need to cheque that no one else has done it first otherwise you could be trying to spend first from something that doesn't exist. If multiple people are trying to delete a row a decision has to be made about the one person who is allowed to do it and the other people will fail. While updating a row you need to be wary of other people updating the same row as depending on the order that updates happen different final outcomes can occur.

For example we have three updates +100 ,-125,+50

Here -125 fails due to low balance and that address is left but in other order if we get +150 first then its a valid transaction.

So every time we are updating a state we need to have an agreed order and should process transactions one at a time. When adding a row we don't need to really check what other people are doing as the order doesn't matter and we can process multiple roles being added in parallel.

HORIZEN ACADEMY

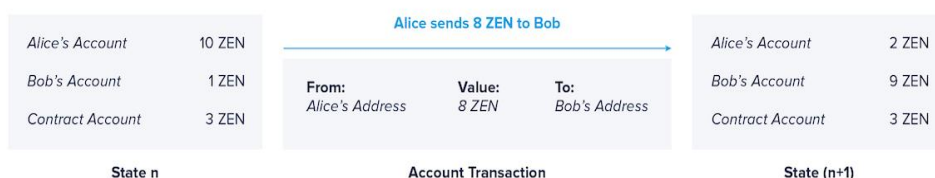


With UTXO model the state is stored as a list of unspent transaction outputs if you look at the same changes transaction you can see that rows are removed and new rows are added. As we are trying to remove rows from the state and there may be multiple people trying to remove the same rows, like with smart contracts consensus is needed for who gets to spend the UTXO, But once that is confirmed you don't need to worry about the order of transactions

UTXO transactions are deterministic either you are able to remove the row needed from the ancient state and the transaction happens or they have already been removed and the transaction fails. This gives a massive advantage for the UTXO to be processed in parallel.

HORIZEN ACADEMY

STATE TRANSITION IN THE ACCOUNT MODEL



In the account model the state is a list of addresses with their balance if we look at the state change for the upon model we can see that instead of adding and removing rows it's updating the rows. As we are updating the rows it requires a transaction to be processed in an agreed order so we cannot process transactions in a parallel like the UTXO model.

The account model is not deterministic; you may or may not be able to update the state depending on the order that the transactions are processed.

If UTXO are more scalable as they run in parallel. Then Why do we need an Account Model ?

The Main reason is smart contracts for smart contracts the simplicity of using an account basis systems makes it much easier for developers Especially if the smart contract needs to interact with many people at once due to there being a updatable state the way the state is stored by address in the account model makes it easier for people as you can simply send a smartcontract script to an address and it will stay there for other people to interact with the state of the Smart contract can be updated when people send clients accidents to the address.

Storing a smart contract stripped in the UTXO model is more difficult as you can't update the state. We can only add or delete in the UTO so more when we store Smart contract script with UTXO model it gets removed when someone interacts with it as UTXO can only be spent once. This means every time you interact with a small contract this means every time you interact with a smart contract script, you need to repost the script in one of the outputs for the next person to use it. It's also a problem if multiple people want to interact with the same smart contract within a block as UTXO can only be spent once per block.